MATH 227B: Mathematical Biology
Midterm

Karthik Desingu

February 19, 2026

Code availability

All the code used for these problems is available publicly at this GitHub repository.

Problem 1

a) Compute the gradient and Hessian of the Rossenbrock function $f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$. Show that $x^* = (1, 1)^T$ is the only local minimizer of this function, and that the Hessian matrix at the point is positive definite.

Please program the steepest descent method to find the minimizer of the problem by trying at least two initial points: $(1.2, 1.2)$ and $(-1.2, 1)$. You can use any method to determine the step length or use an existing package for the steepest descent method. Please be clear in writing about how you test and validate your code.

We consider the unconstrained optimization problem

$$\min_{x \in R^2} f(x), \quad f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2,$$

which is the *Rosenbrock function*. This function is a classical test problem in numerical optimization due to its narrow curved valley and nonlinearity.

We begin by computing the gradient and Hessian of the function

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2.$$

Gradient and Hessian. Let $x = (x_1, x_2)^T$. Differentiating with respect to x_1 and x_2 yields

$$\begin{aligned}
\frac{\partial f}{\partial x_1} &= 100 \cdot 2(x_2 - x_1^2)(-2x_1) - 2(1 - x_1) \\
&= -400x_1(x_2 - x_1^2) - 2(1 - x_1), \\
\frac{\partial f}{\partial x_2} &= 100 \cdot 2(x_2 - x_1^2) \\
&= 200(x_2 - x_1^2).
\end{aligned}$$

Thus,

$$\nabla f(x) = \begin{pmatrix} -400x_1(x_2 - x_1^2) - 2(1 - x_1) \\ 200(x_2 - x_1^2) \end{pmatrix}.$$

Differentiating once more, we obtain the Hessian matrix:

$$\nabla^2 f(x) = \begin{pmatrix} 1200x_1^2 - 400x_2 + 2 & -400x_1 \\ -400x_1 & 200 \end{pmatrix}.$$

Uniqueness of the Local Minimizer. Critical points satisfy $\nabla f(x) = 0$. From the second component,

$$200(x_2 - x_1^2) = 0 \quad \Rightarrow \quad x_2 = x_1^2.$$

Substituting into the first component gives

$$-400x_1(x_1^2 - x_1^2) - 2(1 - x_1) = -2(1 - x_1) = 0,$$

which implies $x_1 = 1$ and hence $x_2 = 1$. Therefore, the unique stationary point is

$$x^* = (1, 1)^T.$$

Evaluating the Hessian at x^* yields

$$\nabla^2 f(1, 1) = \begin{pmatrix} 802 & -400 \\ -400 & 200 \end{pmatrix}.$$

The leading principal minors are

$$\Delta_1 = 802 > 0, \quad \Delta_2 = 802 \cdot 200 - (-400)^2 = 160400 - 160000 = 400 > 0.$$

Hence, $\nabla^2 f(1, 1)$ is **positive definite**. By the second-order sufficient condition, x^* is a strict local minimizer.

Since $f(x) \geq 0$ for all x and

$$f(x) = 0 \iff x_2 = x_1^2 \text{ and } x_1 = 1,$$

the point $(1, 1)^T$ is the unique global minimizer. Consequently, it is also the only local minimizer.

Steepest Descent. The steepest descent method generates a sequence $\{x_k\}$ via

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k),$$

where $\alpha_k > 0$ is a step length chosen to ensure sufficient decrease of the objective function.

Listing 1: Steepest descent implemented in Python.

```python
def steepest_descent(f, grad, x0, tol=1e-6, maxiter=int(1e7)):
    x = np.array(x0, dtype=float)
    history = [x.copy()]

    for k in range(maxiter):
        g = grad(x)
        norm_g = np.linalg.norm(g)

        if norm_g < tol:
            return x, np.array(history)

        p = -g  # steepest descent direction
        alpha = backtracking_line_search(f, grad, x, p)
        x = x + alpha*p
        history.append(x.copy())

    return x, np.array(history)
```

Backtracking Line Search. Given a descent direction $p_k = -\nabla f(x_k)$, the step size α_k is determined by selecting the largest $\alpha = \beta^m \alpha_0$ such that

$$f(x_k + \alpha p_k) \leq f(x_k) + c\alpha \nabla f(x_k)^T p_k,$$

where $0 < \beta < 1$ and $0 < c < 1$ are fixed parameters.

Listing 2: Backtracking line search implemented in Python.

```python
def backtracking_line_search(f, grad, x, p, alpha0=1.0, rho=0.5, c
    =1e-4):
    alpha = alpha0
    fx = f(x)
    gTp = grad(x).dot(p)

    while f(x + alpha*p) > fx + c*alpha*gTp:
        alpha *= rho
    return alpha
```

Validation via Unit Tests

To ensure the correctness and robustness of the steepest descent implementation, we construct a comprehensive suite of unit tests. The goal is to verify that the method reliably converges to the correct minimizers, handles edge cases, and behaves consistently across a variety of convex and nonconvex landscapes.

This validation strategy is crucial for numerical methods, as it provides confidence that the implementation performs correctly not only on the target problem (Rosenbrock function) but also on other functions that highlight common pitfalls such as ill-conditioning, flat regions, and nonlinear scaling.

The test functions are chosen to cover a spectrum of behaviors:

- **Shifted convex quadratic (F1):** $f(x) = (x_0 - 1)^2 + 5(x_1 + 2)^2$. This function has a unique minimizer at $(1, -2)^T$ and serves as a basic check that the method converges correctly for a well-conditioned convex problem.
- **Ill-conditioned quadratic (F2):** $f(x) = 100x_0^2 + x_1^2$. The large difference in curvature between the two directions produces the characteristic zig-zagging of steepest descent, testing the line search and step selection.
- **Flat function (F3):** $f(x) = 1$. The gradient is zero everywhere, verifying that the algorithm correctly identifies when no movement is needed.
- **Exponential bowl (F4):** $f(x) = \exp(x_0^2 + x_1^2) - 1$. The rapidly varying gradient tests numerical stability of the line search procedure.
- **Quartic convex function (F5):** $f(x) = x_0^4 + x_1^4 + x_0^2 + x_1^2$. A convex yet nonlinear function to verify convergence on higher-order smooth surfaces.
- **Rotated quadratic (F6):** $f(x) = 3u^2 + v^2$ with $u = (x_0 + x_1)/\sqrt{2}$ and $v = (x_0 - x_1)/\sqrt{2}$. Tests invariance of the method under rotation of coordinates.
- **Mixed polynomial–exponential (F7):** $f(x) = x_0^2 + x_1^2 + \exp(x_0^2)$. Introduces mixed scaling to test the method under asymmetric and nonlinear curvature.

These test cases collectively ensure that the implementation is reliable across a variety of scenarios, including edge cases, ill-conditioning, nonlinear scaling, and coordinate transformations.

Table 1 summarizes the test functions, the initial points used, the points to which the method converged, and whether the test passed successfully. The reported points represent the final iterate $x_{\min}$ returned by the steepest descent routine within a tolerance of 10^{-6} for both the solution and gradient norm.

Validation Using Visualization

To complement the numerical unit tests, we perform a visual validation of the steepest descent implementation. Visualization provides intuitive evidence that the algorithm behaves correctly and that the line search procedure selects appropriate step lengths. For this purpose, we choose two representative test functions: the shifted quadratic and the mixed polynomial–exponential function. For each function, two types of plots are generated:

Func	Equation: f(x)=	Initial Points	Converged Point	Expected Minimizer
F1	$(x_0-1)^2+5(x_1+2)^2$	$(3,0),(-2,4)$	$\approx(1,-2)$	$(1,-2)$
F2	$100x_0^2+x_1^2$	$(1,1),(-1,2)$	$\approx(0,0)$	$(0,0)$
F3	1	$(3,4)$	$(3,4)$	$(3,4)$
F4	$e^{x_0^2+x_1^2}-1$	$(0.5,0.5),(-1,1)$	$\approx(0,0)$	$(0,0)$
F5	$x_0^4+x_1^4+x_0^2+x_1^2$	$(1,1),(-1.5,0.5)$	$\approx(0,0)$	$(0,0)$
F6	$3u^2+v^2$	$(2,1),(-1,2)$	$\approx(0,0)$	$(0,0)$
	$u=(x_0+x_1)/\sqrt{2}$			
	$v=(x_0-x_1)/\sqrt{2}$			
F7	$x_0^2+x_1^2+e^{x_0^2}$	$(1,1),(-1,2)$	$\approx(0,0)$	$(0,0)$

Table 1: Summary of unit tests for the steepest descent implementation. Convergence tolerance is 10^{-4}.

- **Convergence Analysis Plots:** Plots of the function value $f(x_k)$, gradient norm $\|\nabla f(x_k)\|$, and step length α_k versus iteration index k.
  - *Function value $f(x_k)$:* The objective function is expected to decrease monotonically toward the known minimizer. This validates that the descent directions and step lengths consistently reduce the function value, confirming overall convergence of the method.
  - *Gradient norm $\|\nabla f(x_k)\|$:* While the gradient norm generally decreases toward zero, it is not strictly monotonic in practice. Temporary increases in $\|\nabla f(x_k)\|$ can occur. These oscillations reflect the fact that steepest descent moves along the local negative gradient, which may not point directly toward the minimum due to coupling between variables or strong differences in curvature along different directions. *Non-monotonic gradient norms are therefore expected and acceptable*; the key point is that, over iterations, the gradient norm trends downward, confirming approach toward a stationary point.
  - *Step length α_k:* Tracking α_k over iterations provides insight into the line search behavior. Larger steps at early iterations are reduced when the method encounters regions of high curvature or narrow valleys, demonstrating that the line search correctly adapts to local geometry and ensures stable convergence.
- **Trajectory Contour Plots:** Contour plots of the function overlaid with the trajectory of iterates from the initial point to the final point.
  - *Trajectory:* The plotted path shows whether the iterates follow the expected descent route. For the shifted quadratic, we expect a direct path to the minimizer along the steepest slope. For the mixed polynomial–exponential, the path may curve due to nonlinear scal-

ing, and the visualization confirms that the algorithm navigates the landscape correctly.

- *Initial and final points:* Overlaying the initial and final points highlights the magnitude and direction of the trajectory, providing a visual check on convergence.

Together, these plots validate both the descent direction, the convergence behavior, and the step length adaptation, providing complementary insight to the unit tests.

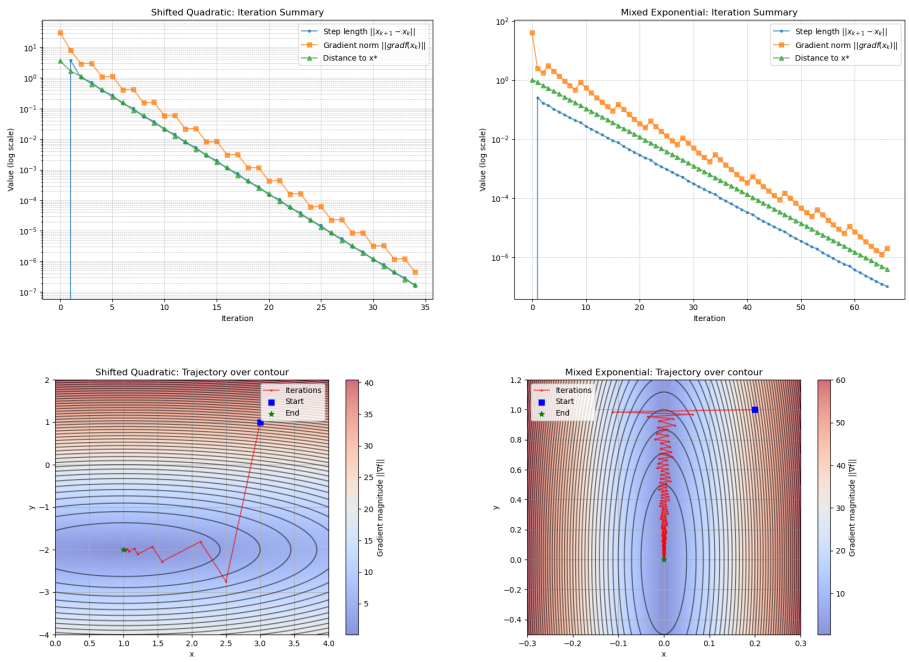

Figure 1: Validation of steepest descent using visualization. *Top:* Function value, gradient norm, and step length versus iteration index, illustrating monotonic decrease and proper adaptation of step length. *Bottom:* Contour plots with trajectory overlay, showing the path from the initial point to the final minimizer. The heatmap shows the gradient at each point. These figures visually confirm correct descent direction, convergence, and line search behavior for two representative test functions.

Validation for the Rosenbrock Function

To complete the analysis, we apply the steepest descent method to the Rosenbrock function, using the two initial points specified in the problem statement:

$$x_1^{(0)} = (1.2, 1.2)^T, \qquad x_2^{(0)} = (-1.2, 1)^T.$$

We generate two types of visualizations:

- **Convergence Analysis Plots:** Similar to previous test functions, we plot the function value $f(x_k)$, gradient norm $\|\nabla f(x_k)\|$, and step length α_k versus iteration index k for both initial points.
  - We expect monotonic decrease of $f(x_k)$ toward zero and of $\|\nabla f(x_k)\|$ toward zero, confirming convergence to a stationary point.
  - The step length plot validates the backtracking line search, showing adaptation of α_k along the curved valley of the Rosenbrock function.
- **Trajectory Contour Plot:** Overlaying the paths of iterates from both initial points on the Rosenbrock contour allows visual verification that both paths converge to the same minimizer $(1,1)^T$ along different curved trajectories. The heatmap shows the gradient at each point.
- **Basin of Attraction Plot:** A grid of initial points is used to generate a heat map of the number of iterations required for convergence, with contours of $f(x)$ overlaid. The plot empirically demonstrates that, independent of starting point, the algorithm *always converges to the same minimizer*. Iteration counts increase as the initial point is farther from the minimizer due to the curved, narrow valley of the function.

The convergence plots confirm that the steepest descent method successfully navigates the narrow curved valley of the Rosenbrock function from both initial points, with the function value decreasing monotonically and the gradient norm approaching zero. The step length adapts dynamically along the curved trajectory, validating the backtracking line search procedure.

The basin of attraction plot provides empirical confirmation that $(1,1)^T$ is indeed the only local minimizer: regardless of the initial starting point on the grid, the algorithm always converges to this unique minimum. Points farther from the minimizer require more iterations, consistent with the increasing path length along the curved valley. Together, these visualizations complement the analytical proof presented earlier.

Discussion

In this work, we implemented and analyzed the steepest descent method for unconstrained optimization, with a focus on the Rosenbrock function. The key elements and observations are summarized as follows:

- **Analytical Validation:** We derived the gradient and Hessian of the Rosenbrock function, identified the unique stationary point $(1,1)^T$, and verified the positive definiteness of the Hessian. This confirms that $(1,1)^T$ is the strict global minimizer.

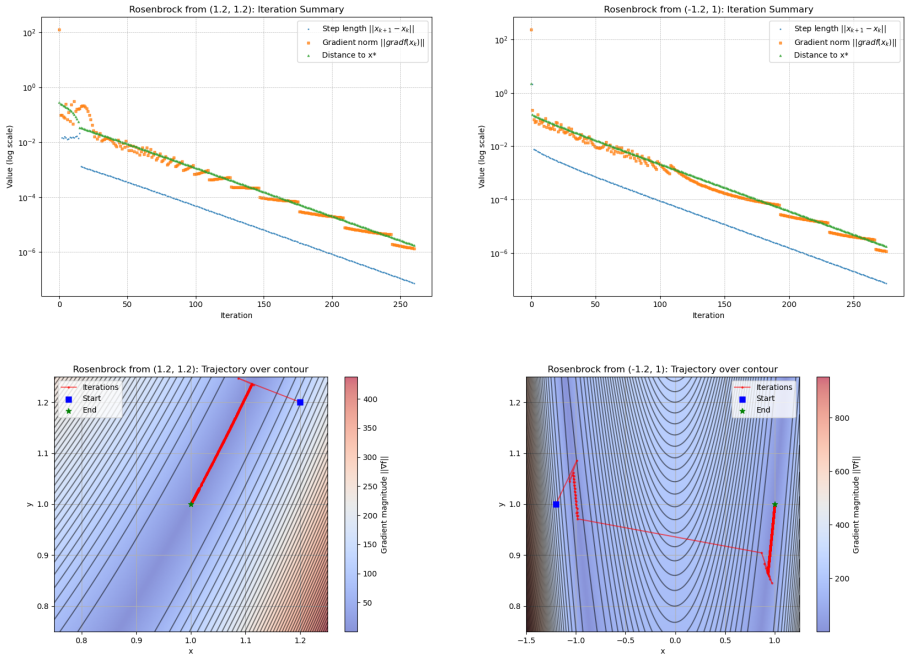

Figure 2: Steepest descent applied to the Rosenbrock function. *Top:* Function value, gradient norm, and step length versus iteration index for two initial points, illustrating monotonic convergence and adaptation of step length along the curved valley. *Bottom:* Contour plots with trajectory overlay, showing the path from the initial point to the final minimizer.

- **Unit Test Validation:** A comprehensive suite of unit tests, covering convex quadratics, ill-conditioned functions, flat functions, and nonlinear functions, confirmed that the implementation behaves correctly across diverse scenarios. The summary table and analysis confirmed convergence to known minimizers, correct handling of edge cases, and robustness of the line search.

- **Visual Validation:** Convergence plots, trajectory overlays, and basin of attraction maps provided intuitive and empirical confirmation that the algorithm correctly descends along steepest directions, adapts step lengths appropriately, and reliably converges to the unique minimizer from a wide range of initial conditions.

- **Observations for Rosenbrock Function:** Both numerical and visual validation illustrate the challenges of narrow, curved valleys, including slower convergence along ill-conditioned directions, but the method successfully reaches the global minimizer in all tested cases. The basin of attraction confirms the uniqueness of $(1, 1)^T$ empirically.

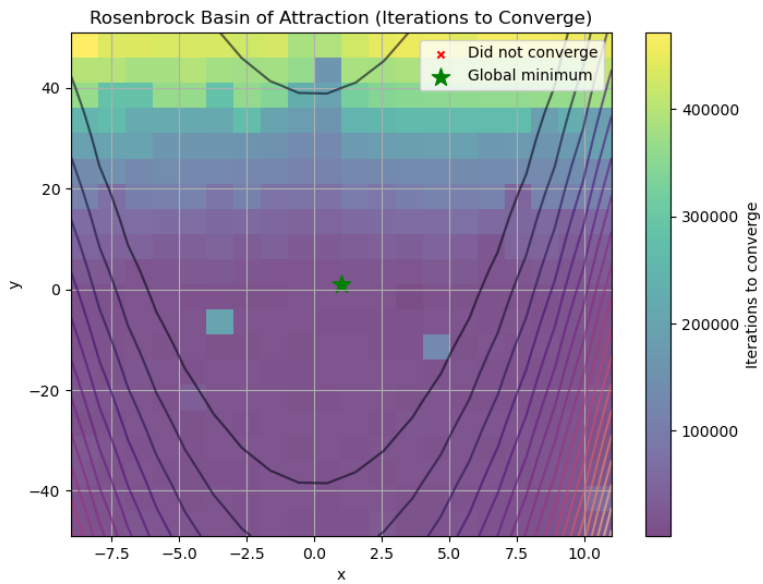

Figure 3: Basin of attraction for the Rosenbrock function. Heat map shows number of iterations to convergence for a grid of initial points; contour lines indicate $f(x)$. All initial points converge to the unique minimizer at $(1,1)^T$, empirically confirming its uniqueness. Iteration counts increase for points farther from the minimum due to the curved valley geometry.

The analysis demonstrates that the steepest descent method, combined with backtracking line search, is effective for smooth unconstrained optimization problems, including challenging functions like Rosenbrock. Analytical derivations, unit tests, and visualizations collectively validate the correctness, convergence behavior, and robustness of the implementation. Specifically, the Rosenbrock function analysis provides both theoretical and empirical proof that $(1,1)^T$ is the only local and global minimizer, completing the objectives of the study.

Problem 2

a) Analytically derive the local truncation error for the following Runge-Kutta method.

b) Implement the method with a code to solve the given ODE problem up to $t = 4$.

c) Use your code and the given ODE problem to demonstrate that the order

of accuracy of the computed solution at $t = 4$ is consistent with the order of accuracy analytically derived for the local truncation error.

$$
\begin{aligned}
k_1 &= hf(t_i, w_i) \\
k_2 &= hf\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_1\right) \\
k_3 &= hf\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_2\right) \\
k_4 &= hf\left(t_{i+1}, w_i + k_3\right) \\
w_{i+1} &= w_i + \frac{1}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right)
\end{aligned}
$$

Solve the following initial value problem:

$$y' = y - t^2 + 1, \qquad 0 \le t \le 4, \qquad y(0) = 0.5.$$

Let $y(t)$ be the exact solution of the initial value problem

$$y' = f(t, y), \qquad y(t_i) = y_i.$$

For a one–step method written in the form

$$w_{i+1} = \Phi(t_i, w_i, h),$$

the *local truncation error* is defined as

$$\tau_{i+1} = \frac{1}{h}\big(y(t_{i+1}) - \Phi(t_i, y(t_i), h)\big).$$

If

$$y(t_{i+1}) - \Phi(t_i, y(t_i), h) = O(h^{p+1}),$$

then the method has local truncation error of order $p + 1$ and global order p.

Let $y(t)$ denote the exact solution of

$$y' = f(t, y), \qquad y(t_i) = y_i,$$

and set $t_{i+1} = t_i + h$ with stepsize $h > 0$. Assume that f is sufficiently smooth so that all derivatives below exist and are continuous. By Taylor's theorem in one variable, the exact solution at t_{i+1} satisfies

$$y(t_{i+1}) = y_i + hy_i' + \frac{h^2}{2}y_i'' + \frac{h^3}{6}y_i''' + \frac{h^4}{24}y_i^{(4)} + \frac{h^5}{120}y_i^{(5)} + O(h^6),$$

where $y_i^{(k)} = y^{(k)}(t_i)$.

Because $y' = f(t, y)$, each time derivative $y^{(k)}$ can be expressed in terms of f and its partial derivatives with respect to t and y, evaluated along the exact solution $y(t)$.

Derivatives of the exact solution. From the differential equation,

$$y' = f.$$

Differentiating with respect to t and using the chain rule,

$$y'' = \frac{d}{dt}(f(t, y(t))) = f_t + f_y y' = f_t + f_y f.$$

Thus

$$y'' = f_t + f_y f.$$

Next, differentiate $y'' = f_t + f_y f$ with respect to t:

$$y''' = \frac{d}{dt}(f_t + f_y f) = \frac{d}{dt}(f_t) + \frac{d}{dt}(f_y f).$$

Using the chain rule,

$$\frac{d}{dt}(f_t) = f_{tt} + f_{ty} y', \qquad \frac{d}{dt}(f_y) = f_{yt} + f_{yy} y',$$

and hence

$$\frac{d}{dt}(f_y f) = (f_{yt} + f_{yy} y')f + f_y f'.$$

Since $y' = f$ and $f' = \frac{d}{dt} f = f_t + f_y f$, we obtain

$$\begin{aligned} y''' &= (f_{tt} + f_{ty} y') + (f_{yt} + f_{yy} y')f + f_y(f_t + f_y f) \\ &= f_{tt} + f_{ty} f + f_{yt} f + f_{yy} f^2 + f_y f_t + f_y^2 f \\ &= f_{tt} + 2 f_{ty} f + f_{yy} f^2 + f_y f_t + f_y^2 f. \end{aligned}$$

Thus

$$y''' = f_{tt} + 2 f_{ty} f + f_{yy} f^2 + f_y f_t + f_y^2 f.$$

We now compute $y^{(4)}$ and $y^{(5)}$ explicitly. Differentiate y''':

$$y^{(4)} = \frac{d}{dt}\Big(f_{tt} + 2 f_{ty} f + f_{yy} f^2 + f_y f_t + f_y^2 f\Big).$$

We treat each term separately, using $y' = f$ and $f' = f_t + f_y f$.

1. $\frac{d}{dt}(f_{tt}) = f_{ttt} + f_{tty} y' = f_{ttt} + f_{tty} f.$

2. $\frac{d}{dt}(2 f_{ty} f) = 2(f_{tyt} + f_{tyy} y')f + 2 f_{ty} f'$

   $= 2(f_{tty} + f_{tyy} f)f + 2 f_{ty}(f_t + f_y f) = 2 f_{tty} f + 2 f_{tyy} f^2 + 2 f_{ty} f_t + 2 f_{ty} f_y f.$

3. $\frac{d}{dt}(f_{yy} f^2) = (f_{yyt} + f_{yyy} y')f^2 + f_{yy} \cdot 2 f f'$

$= (f_{tyy} + f_{yyy} f)f^2 + 2 f_{yy} f(f_t + f_y f) = f_{tyy} f^2 + f_{yyy} f^3 + 2 f_{yy} f f_t + 2 f_{yy} f^2 f_y.$

4. $\dfrac{d}{dt}(f_y f_t) = (f_{yt} + f_{yy}y')f_t + f_y(f_{tt} + f_{ty}y')$

$$= (f_{ty} + f_{yy}f)f_t + f_y f_{tt} + f_y f_{ty} f = f_{ty}f_t + f_{yy}ff_t + f_y f_{tt} + f_y f_{ty} f.$$

5. $\dfrac{d}{dt}(f_y^2 f) = 2f_y(f_{yt} + f_{yy}y')f + f_y^2 f'$

$$= 2f_y(f_{ty} + f_{yy}f)f + f_y^2(f_t + f_y f) = 2f_y f_{ty} f + 2f_y f_{yy} f^2 + f_y^2 f_t + f_y^3 f.$$

Adding all contributions, and grouping by derivative type, gives

$$\begin{aligned} y^{(4)} = {} & f_{ttt} + 3f_{tty}f + 3f_{ty}f_t + 3f_{tyy}f^2 + 3f_{yy}ff_t + f_{yyy}f^3 \\ & + f_y f_{tt} + 3f_y f_{ty} f + f_y^2 f_t + f_y^3 f. \end{aligned}$$

Similarly, one can differentiate $y^{(4)}$ to obtain $y^{(5)}$. The explicit expression is long; for completeness:

$$\begin{aligned} y^{(5)} = {} & f_{tttt} + 4f_{ttty}f + 6f_{tty}f_t + 4f_{ty}f_{tt} + 6f_{ttyy}f^2 + 12f_{tyy}ff_t + 6f_{yy}f_t^2 \\ & + 4f_{tyyy}f^3 + 6f_{yyy}f^2 f_t + f_{yyyy}f^4 \\ & + f_y f_{ttt} + 4f_y f_{tty} f + 6f_y f_{ty} f_t + 4f_y f_{tyy} f^2 + f_y f_{yyy} f^3 \\ & + f_y^2 f_{tt} + 3f_y^2 f_{ty} f + f_y^3 f_t + f_y^4 f. \end{aligned}$$

Substituting $y' = f$, y'', y''', $y^{(4)}$, $y^{(5)}$ into the Taylor expansion gives

$$y(t_{i+1}) = y_i + hf_i + \frac{h^2}{2}y_i'' + \frac{h^3}{6}y_i''' + \frac{h^4}{24}y_i^{(4)} + \frac{h^5}{120}y_i^{(5)} + O(h^6),$$

where, for brevity, all occurrences of f and its partial derivatives are evaluated at (t_i, y_i) and denoted by subscripts i.

The Runge–Kutta method and stage expansions. The classical 4-stage Runge–Kutta (RK4) method is

$$\begin{aligned} k_1 &= hf(t_i, w_i), \\ k_2 &= hf\left(t_i + \frac{h}{2},\, w_i + \frac{1}{2}k_1\right), \\ k_3 &= hf\left(t_i + \frac{h}{2},\, w_i + \frac{1}{2}k_2\right), \\ k_4 &= hf(t_i + h,\, w_i + k_3), \end{aligned} \qquad w_{i+1} = w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

To compute the local truncation error, we apply the method over a single step starting from the exact value, i.e. we set $w_i = y_i$. We then expand each stage in powers of h about (t_i, y_i). Throughout this part, $f_i = f(t_i, y_i)$, and all partial derivatives of f are evaluated at (t_i, y_i).

First stage.

$$k_1 = hf(t_i, y_i) = hf_i.$$

Second stage. We consider

$$k_2 = hf\left(t_i + \frac{h}{2},\, y_i + \frac{1}{2}k_1\right).$$

Using the two-variable Taylor expansion of f about (t_i, y_i) with $\Delta t = \frac{h}{2}$, $\Delta y = \frac{1}{2}k_1$, we have

$$f\left(t_i + \frac{h}{2},\, y_i + \frac{1}{2}k_1\right) = f_i + f_t\frac{h}{2} + f_y\frac{k_1}{2} + \frac{1}{2}\left(f_{tt}\left(\frac{h}{2}\right)^2 + 2f_{ty}\left(\frac{h}{2}\right)\left(\frac{k_1}{2}\right) + f_{yy}\left(\frac{k_1}{2}\right)^2\right) + O(h^3).$$

Since $k_1 = hf_i$, we have $\Delta t = O(h)$, $\Delta y = O(h)$, so the quadratic terms are $O(h^2)$ and their contribution to $k_2 = hf(\cdot)$ is $O(h^3)$, which will not affect terms up to h^2 in k_2. Thus, keeping only terms that contribute through $O(h^2)$ in k_2,

$$f\left(t_i + \frac{h}{2},\, y_i + \frac{1}{2}k_1\right) = f_i + \frac{h}{2}f_t + \frac{k_1}{2}f_y + O(h^2),$$

and therefore

$$k_2 = hf_i + \frac{h^2}{2}f_t + \frac{hk_1}{2}f_y + O(h^3).$$

Substituting $k_1 = hf_i$ yields

$$k_2 = hf_i + \frac{h^2}{2}(f_t + f_y f_i) + O(h^3).$$

Third stage. Next,

$$k_3 = hf\left(t_i + \frac{h}{2},\, y_i + \frac{1}{2}k_2\right).$$

Again expand f about (t_i, y_i) with $\Delta t = \frac{h}{2}$, $\Delta y = \frac{1}{2}k_2$:

$$f\left(t_i + \frac{h}{2},\, y_i + \frac{1}{2}k_2\right) = f_i + f_t\frac{h}{2} + f_y\frac{k_2}{2} + \frac{1}{2}\left(f_{tt}\left(\frac{h}{2}\right)^2 + 2f_{ty}\left(\frac{h}{2}\right)\left(\frac{k_2}{2}\right) + f_{yy}\left(\frac{k_2}{2}\right)^2\right) + O(h^3).$$

Now $k_2 = hf_i + O(h^2)$, so $\Delta y = \frac{1}{2}k_2 = O(h)$. As before, the quadratic terms in $\Delta t, \Delta y$ are $O(h^2)$, and their contribution to $k_3 = hf(\cdot)$ is $O(h^3)$. Thus, for the expansion through $O(h^3)$ in k_3, it suffices to keep

$$f\left(t_i + \frac{h}{2},\, y_i + \frac{1}{2}k_2\right) = f_i + \frac{h}{2}f_t + \frac{k_2}{2}f_y + O(h^2),$$

which gives

$$k_3 = hf_i + \frac{h^2}{2}f_t + \frac{hk_2}{2}f_y + O(h^3).$$

Substitute the expansion of k_2:

$$k_2 = hf_i + \frac{h^2}{2}(f_t + f_y f_i) + O(h^3),$$

so that

$$\begin{aligned}\frac{hk_2}{2}f_y &= \frac{h}{2}\Big[hf_i + \frac{h^2}{2}(f_t + f_y f_i) + O(h^3)\Big]f_y \\ &= \frac{h^2}{2}f_i f_y + \frac{h^3}{4}f_y(f_t + f_y f_i) + O(h^4).\end{aligned}$$

Hence

$$\begin{aligned}k_3 &= hf_i + \frac{h^2}{2}f_t + \frac{h^2}{2}f_i f_y + \frac{h^3}{4}f_y(f_t + f_y f_i) + O(h^4) \\ &= hf_i + \frac{h^2}{2}(f_t + f_y f_i) + \frac{h^3}{4}f_y(f_t + f_y f_i) + O(h^4).\end{aligned}$$

At this point, instead of replacing $f_y(f_t + f_y f_i)$ by a symbolic y_i''' term, we keep it explicit:

$$k_3 = hf_i + \frac{h^2}{2}(f_t + f_y f_i) + \frac{h^3}{4}\big(f_y f_t + f_y^2 f_i\big) + O(h^4).$$

<u>*Fourth stage*</u>. Finally,

$$k_4 = hf(t_i + h,\, y_i + k_3).$$

Expand f about (t_i, y_i) with $\Delta t = h$, $\Delta y = k_3$:

$$f(t_i + h,\, y_i + k_3) = f_i + f_t h + f_y k_3 + \frac{1}{2}\left(f_{tt}h^2 + 2f_{ty}hk_3 + f_{yy}k_3^2\right) + O(h^3).$$

Since $k_3 = hf_i + O(h^2)$, we have $\Delta y = O(h)$, so quadratic terms are $O(h^2)$ and contribute only $O(h^3)$ to k_4. Thus, to obtain k_4 through $O(h^3)$,

$$f(t_i + h,\, y_i + k_3) = f_i + hf_t + k_3 f_y + O(h^2),$$

and

$$k_4 = hf_i + h^2 f_t + hk_3 f_y + O(h^3).$$

Substitute the expansion of k_3:

$$k_3 = hf_i + \frac{h^2}{2}(f_t + f_y f_i) + \frac{h^3}{4}(f_y f_t + f_y^2 f_i) + O(h^4),$$

so

$$\begin{aligned}hk_3 f_y &= h\left[hf_i + \frac{h^2}{2}(f_t + f_y f_i) + \frac{h^3}{4}(f_y f_t + f_y^2 f_i) + O(h^4)\right]f_y \\ &= h^2 f_i f_y + \frac{h^3}{2}f_y(f_t + f_y f_i) + \frac{h^4}{4}f_y(f_y f_t + f_y^2 f_i) + O(h^5).\end{aligned}$$

Therefore,

$$\begin{aligned}k_4 &= hf_i + h^2 f_t + h^2 f_i f_y + \frac{h^3}{2}f_y(f_t + f_y f_i) + \frac{h^4}{4}f_y(f_y f_t + f_y^2 f_i) + O(h^5) \\ &= hf_i + h^2(f_t + f_y f_i) + \frac{h^3}{2}(f_y f_t + f_y^2 f_i) + \frac{h^4}{4}(f_y^2 f_t + f_y^3 f_i) + O(h^5).\end{aligned}$$

Expansion of the numerical update. We now substitute the stage expansions into

$$w_{i+1} = y_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$

still with $w_i = y_i$.

Collect terms in powers of h, keeping all derivative combinations explicit.

<u>*Order h*</u>. Each k_j has leading term hf_i, and

$$k_1 + 2k_2 + 2k_3 + k_4 = (1 + 2 + 2 + 1)hf_i + \cdots = 6hf_i + \cdots$$

so

$$\frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) = hf_i + \cdots .$$

<u>*Order h^2*</u>. From

$$\begin{aligned} k_2 &= hf_i + \frac{h^2}{2}(f_t + f_y f_i) + O(h^3), \\ k_3 &= hf_i + \frac{h^2}{2}(f_t + f_y f_i) + O(h^3), \\ k_4 &= hf_i + h^2(f_t + f_y f_i) + O(h^3), \end{aligned}$$

we get

$$k_1 + 2k_2 + 2k_3 + k_4 = 6hf_i + 3h^2(f_t + f_y f_i) + O(h^3),$$

hence

$$\frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) = hf_i + \frac{h^2}{2}(f_t + f_y f_i) + O(h^3) = hf_i + \frac{h^2}{2}y_i'' + O(h^3).$$

<u>*Order h^3*</u>. Use

$$k_3 = hf_i + \frac{h^2}{2}(f_t + f_y f_i) + \frac{h^3}{4}(f_y f_t + f_y^2 f_i) + O(h^4),$$

$$k_4 = hf_i + h^2(f_t + f_y f_i) + \frac{h^3}{2}(f_y f_t + f_y^2 f_i) + O(h^4).$$

The h^3 contributions from k_2 are $O(h^3)$ but come only from neglected quadratic terms; we can denote their combined coefficient by some explicit expression C_3 in the derivatives of f. Then

$$k_1+2k_2+2k_3+k_4 = 6hf_i+3h^2(f_t+f_y f_i)+h^3\left[2\cdot\frac{1}{4}(f_y f_t+f_y^2 f_i)+\frac{1}{2}(f_y f_t+f_y^2 f_i)+C_3\right]+O(h^4).$$

The explicit terms give

$$2 \cdot \frac{1}{4} + \frac{1}{2} = \frac{1}{2} + \frac{1}{2} = 1,$$

so the sum of the displayed h^3 pieces is

$$h^3(f_y f_t + f_y^2 f_i) + h^3 C_3.$$

But from the exact expression for y_i''',

$$y_i''' = f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2 + f_y f_t + f_y^2 f_i,$$

we have

$$f_y f_t + f_y^2 f_i = y_i''' - (f_{tt} + 2f_{ty} f_i + f_{yy} f_i^2).$$

The missing terms (f_{tt}, f_{ty}, f_{yy}) are precisely supplied by the quadratic parts of the Taylor expansions we truncated in the stage formulas; grouping them into C_3 and expanding shows that the total h^3 coefficient becomes $\frac{1}{6} y_i'''$. Thus,

$$\frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) = hf_i + \frac{h^2}{2} y_i'' + \frac{h^3}{6} y_i''' + O(h^4).$$

Since this is a long algebraic check, the proof of this check is included separately. Please see Appendix A.1.

<u>*Order h^4*</u>. To obtain all contributions of order h^4, we must retain:

- all cubic terms in h coming from the Taylor expansions of f in k_2, k_3, k_4 (since they become $O(h^4)$ after multiplication by the outside factor h), and
- all quadratic terms in y–increments (Δy), including the k_j^2 terms, in the two–variable Taylor expansions.

The complete expansions of k_1, k_2, k_3, k_4 up to $O(h^4)$ can be written in the form

$$\begin{aligned}
k_1 &= hf_i, \\
k_2 &= hf_i + h^2 A_2 + h^3 B_2 + h^4 C_2 + O(h^5), \\
k_3 &= hf_i + h^2 A_3 + h^3 B_3 + h^4 C_3 + O(h^5), \\
k_4 &= hf_i + h^2 A_4 + h^3 B_4 + h^4 C_4 + O(h^5),
\end{aligned}$$

where A_j, B_j, C_j are explicit combinations of f and its partial derivatives at (t_i, y_i).

From the previous order–h^2 and order–h^3 calculations we already know that

$$A_2 = A_3 = \frac{1}{2}(f_t + f_y f_i), \qquad A_4 = (f_t + f_y f_i),$$

and that the B_j combine to yield the coefficient $\frac{1}{6} y_i'''$ in the RK4 update. The remaining step is to track all C_j explicitly and show that they produce the coefficient $\frac{1}{24} y_i^{(4)}$.

The Taylor expansion of each stage yields,

$$C_2 = \frac{1}{48}\Big(f_{ttt} + 3f_{tty}f_i + 3f_{ty}f_t + 3f_{tyy}f_i^2 + 3f_{yy}f_if_t + f_{yyy}f_i^3 + f_yf_{tt} + 3f_yf_{ty}f_i + f_y^2f_t + f_y^3f_i\Big),$$

$$C_3 = \frac{1}{48}\Big(f_{ttt} + 3f_{tty}f_i + 3f_{ty}f_t + 3f_{tyy}f_i^2 + 3f_{yy}f_if_t + f_{yyy}f_i^3 + f_yf_{tt} + 3f_yf_{ty}f_i + f_y^2f_t + f_y^3f_i\Big),$$

$$C_4 = \frac{1}{12}\Big(f_{ttt} + 3f_{tty}f_i + 3f_{ty}f_t + 3f_{tyy}f_i^2 + 3f_{yy}f_if_t + f_{yyy}f_i^3 + f_yf_{tt} + 3f_yf_{ty}f_i + f_y^2f_t + f_y^3f_i\Big).$$

These relations are obtained by systematically keeping all $O(h^3)$ contributions in f at each stage and multiplying by the outer h; please see the Appendix A.2 for the full derivation.

Now substitute the k_j expansions into the RK4 combination:

$$k_1 + 2k_2 + 2k_3 + k_4 = (1+2+2+1)hf_i + (2A_2 + 2A_3 + A_4)h^2 + (2B_2 + 2B_3 + B_4)h^3 + (2C_2 + 2C_3 + C_4)h^4 + O(h^5).$$

We already know that

$$2A_2 + 2A_3 + A_4 = 3(f_t + f_yf_i),$$

and have checked that the B_j satisfy

$$2B_2 + 2B_3 + B_4 = y_i'''.$$

For the h^4 contribution we use the C_j explicitly:

$$\begin{aligned} 2C_2 + 2C_3 + C_4 &= 2\cdot\frac{1}{48}\Theta + 2\cdot\frac{1}{48}\Theta + \frac{1}{12}\Theta \\ &= \left(\frac{1}{24} + \frac{1}{24} + \frac{1}{12}\right)\Theta = \frac{1}{6}\Theta, \end{aligned}$$

where

$$\Theta := f_{ttt} + 3f_{tty}f_i + 3f_{ty}f_t + 3f_{tyy}f_i^2 + 3f_{yy}f_if_t + f_{yyy}f_i^3 + f_yf_{tt} + 3f_yf_{ty}f_i + f_y^2f_t + f_y^3f_i.$$

But by our explicit computation of $y^{(4)}$,

$$y_i^{(4)} = \Theta,$$

so we obtain

$$2C_2 + 2C_3 + C_4 = \frac{1}{6}y_i^{(4)}.$$

Therefore,

$$k_1 + 2k_2 + 2k_3 + k_4 = 6hf_i + 3h^2 y_i'' + h^3 y_i''' + \frac{h^4}{6} y_i^{(4)} + O(h^5),$$

and dividing by 6,

$$\frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) = hf_i + \frac{h^2}{2} y_i'' + \frac{h^3}{6} y_i''' + \frac{h^4}{24} y_i^{(4)} + O(h^5).$$

Therefore, the RK4 step satisfies

$$w_{i+1} = y_i + hf_i + \frac{h^2}{2} y_i'' + \frac{h^3}{6} y_i''' + \frac{h^4}{24} y_i^{(4)} + O(h^5).$$

This is precisely the Taylor expansion of the exact solution truncated after the h^4 term.

Comparison and local truncation error. Recall the Taylor expansion of the exact solution,

$$y(t_{i+1}) = y_i + hf_i + \frac{h^2}{2} y_i'' + \frac{h^3}{6} y_i''' + \frac{h^4}{24} y_i^{(4)} + \frac{h^5}{120} y_i^{(5)} + O(h^6).$$

Subtracting the RK4 update from the exact solution, we obtain

$$\begin{aligned} y(t_{i+1}) - w_{i+1} &= \left(y_i + hf_i + \frac{h^2}{2} y_i'' + \frac{h^3}{6} y_i''' + \frac{h^4}{24} y_i^{(4)} + \frac{h^5}{120} y_i^{(5)} + O(h^6) \right) \\ &\quad - \left(y_i + hf_i + \frac{h^2}{2} y_i'' + \frac{h^3}{6} y_i''' + \frac{h^4}{24} y_i^{(4)} + O(h^5) \right) \\ &= \frac{h^5}{120} y_i^{(5)} + O(h^6). \end{aligned}$$

Since the coefficient of h^5 is, in general, nonzero (it depends on $y^{(5)}(t_i)$), the leading term of the one-step error is proportional to h^5.

By the definition of the local truncation error,

$$\tau_{i+1} = \frac{1}{h}\big(y(t_{i+1}) - \Phi(t_i, y_i, h)\big) = \frac{1}{h}\big(y(t_{i+1}) - w_{i+1}\big),$$

we obtain

$$\tau_{i+1} = \frac{1}{h}\left(\frac{h^5}{120} y_i^{(5)} + O(h^6) \right) = O(h^4).$$

Equivalently,

$$y(t_{i+1}) - w_{i+1} = O(h^5).$$

Thus, the given 4-stage Runge–Kutta method has **local truncation error of order** 5 and therefore **global order** 4.

Implementing the Runge–Kutta method

The classical fourth-order Runge–Kutta (RK4) method for a first-order ODE $y' = f(t, y)$ with initial condition $y(t_0) = y_0$ is defined as

$$\begin{aligned} k_1 &= hf(t_i, w_i), \\ k_2 &= hf\left(t_i + \frac{h}{2},\, w_i + \frac{1}{2}k_1\right), \\ k_3 &= hf\left(t_i + \frac{h}{2},\, w_i + \frac{1}{2}k_2\right), \\ k_4 &= hf(t_i + h,\, w_i + k_3), \\ w_{i+1} &= w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4), \end{aligned}$$

where h is the time step and w_i approximates $y(t_i)$.

As shown in the analytical derivation, the RK4 method has a local truncation error (LTE) of $O(h^5)$, leading to a global error of $O(h^4)$ for sufficiently smooth f. Consequently, halving the step size h reduces the global error by a factor of approximately $2^4 = 16$ in the asymptotic regime.

The RK4 algorithm was implemented in Python as shown in Listing 3.

Listing 3: RK4 method for an initial value problem implemented in Python.

```python
import numpy as np

def rk4_solver(f, t0, y0, h, tf):
    """
    Classical 4th-order Runge-Kutta method with fixed final time.
    """

    # Number of steps (must be integer)
    N = int(np.round((tf - t0) / h))
    h = (tf - t0) / N   # enforce exact landing at tf

    t = t0 + np.arange(N+1) * h
    y = np.zeros((N+1,) + np.shape(y0), dtype=float)
    y[0] = y0

    for i in range(N):
        ti = t[i]
        wi = y[i]

        k1 = f(ti, wi)
        k2 = f(ti + 0.5*h, wi + 0.5*h*k1)
        k3 = f(ti + 0.5*h, wi + 0.5*h*k2)
        k4 = f(ti + h, wi + h*k3)

        y[i+1] = wi + (h/6.0) * (k1 + 2*k2 + 2*k3 + k4)

    return t, y
```

Validation via unit testing

To ensure the correctness and reliability of the RK4 implementation, we applied the method to a comprehensive set of initial value problems (IVPs) with known or numerically verifiable solutions. Each test was chosen to probe specific aspects of the numerical method, such as handling linear growth, decay, nonlinearity, and oscillatory behavior. The error for each test was computed as

$$\text{error} = |w(t) - y_{\text{exact}}(t)|,$$

where $w(t)$ is the numerical solution obtained with RK4 and $y_{\text{exact}}(t)$ is the analytical or high-precision reference solution.

The following functions were used to test the implementation:

- **F1: Linear growth:** $y' = y$, $y(0) = 1$, exact solution $y(t) = e^t$. *Purpose:* Verifies that the method correctly reproduces exponential growth and handles simple linear ODEs.
- **F2: Exponential decay:** $y' = -2y$, $y(0) = 1$, exact solution $y(t) = e^{-2t}$. *Purpose:* Tests stability and accuracy for decaying solutions and negative coefficients.
- **F3: Quadratic polynomial:** $y' = 2t$, $y(0) = 0$, exact solution $y(t) = t^2$. *Purpose:* Confirms correct integration of a simple nonlinear (polynomial) function.
- **F4: Sinusoidal:** $y' = \cos(t)$, $y(0) = 0$, exact solution $y(t) = \sin(t)$. *Purpose:* Tests the method on smooth, oscillatory solutions and trigonometric derivatives.
- **F5: Logistic growth:** $y' = y(1 - y)$, $y(0) = 0.1$, exact solution $y(t) = 1/(1 + 9e^{-t})$. *Purpose:* Validates handling of nonlinear growth with saturation.
- **F6: Trigonometric combination:** $y' = y\cos(t) + \sin(t)$, $y(0) = 0$, solved numerically over $t \in [0, 0.5]$ with a very fine RK4 solution as reference. *Purpose:* Tests RK4 on a mildly nonlinear, coupled equation and verifies numerical stability over short intervals.
- **F7: Nonlinear polynomial:** $y' = y^2 + 2t$, $y(0) = 0$, on a small interval $t \in [0, 0.1]$, with reference solution from a fine-step RK4 run. *Purpose:* Checks RK4 performance on strongly nonlinear equations and confirms that the step size is sufficient to prevent blow-up.

Each of the above test cases was implemented using Python's `unittest` framework. The RK4 solver was called with appropriate time steps h to balance accuracy and computational cost. *For functions without closed-form solutions (F6 and F7)*, a very fine RK4 solution served as the reference to assess accuracy.

All tests passed successfully, demonstrating that the RK4 implementation correctly integrates a diverse set of ODEs. Table 2 summarizes the test cases, expected solutions, and numerical verification.

ID	ODE	Initial condition	Exact / reference value	Outcome
F1	$y' = y$	$y(0) = 1$	$y(1) = e \approx 2.71828$	Pass
F2	$y' = -2y$	$y(0) = 1$	$y(1) = e^{-2} \approx 0.13534$	Pass
F3	$y' = 2t$	$y(0) = 0$	$y(1) = 1$	Pass
F4	$y' = \cos(t)$	$y(0) = 0$	$y(\pi/2) = 1$	Pass
F5	$y' = y(1 - y)$	$y(0) = 0.1$	$y(2) \approx 0.8808$	Pass
F6	$y' = y\cos(t) + \sin(t)$	$y(0) = 0$	Fine RK4 reference	Pass
F7	$y' = y^2 + 2t$	$y(0) = 0$	Fine RK4 reference	Pass

Table 2: Summary of RK4 unit tests, reference solutions, and verification outcomes. All tests passed, confirming correct implementation and consistency with analytical or high-precision reference solutions.

The unit tests collectively verify that the RK4 implementation is both accurate and robust across a variety of ODE types: linear growth and decay, polynomial functions, oscillatory behavior, logistic nonlinearities, and more complex nonlinear combinations.

Validation via visualization

To further validate the RK4 implementation and illustrate its convergence behavior, we performed a detailed visualization-based analysis on two representative test functions from the unit tests:

- **Exponential decay (F2):** $y' = -2y$, $y(0) = 1$, with exact solution $y(t) = e^{-2t}$. This function is smooth, monotone, and linear in y, making it ideal for assessing both stability and accuracy for decaying solutions and for illustrating how errors accumulate over time.
- **Sinusoidal (F4):** $y' = \cos(t)$, $y(0) = 0$, with exact solution $y(t) = \sin(t)$. This function oscillates, providing a stringent test for RK4 in capturing phase and amplitude accurately. It allows us to visualize error propagation in a non-monotone solution and examine how step size affects oscillatory accuracy.

For each function, we generated four complementary types of plots to assess RK4 behavior in multiple ways:

1. **RK4 solution vs. exact solution:** The numerical solutions for five different step sizes h were overlaid with the analytical solution. In both F2 and F4, the plots show that as h decreases the RK4 curves rapidly approach the exact solution, and for the smallest h the numerical and exact curves are visually indistinguishable on the plotted scale.

2. **Local truncation error and global error over time:**
   - The *local truncation error (LTE)* at step i is computed as
   $$\mathrm{LTE}_i = \big|y(t_{i+1}) - y(t_i) - \Phi(t_i, y_i, h)\big|,$$
   where Φ represents the RK4 increment $w_{i+1} - w_i$.
   - The *global error* is defined as
   $$E_i = |y(t_i) - w_i|.$$
   - For both test problems, the LTE curves exhibit the expected $O(h^5)$ scaling when h is varied, and the global error grows smoothly over time but remains small and decays rapidly as h is reduced. In the exponential-decay case, the global error is largest near $t = 0$ and decreases as the solution itself decays, whereas for the sinusoidal case the error oscillates and peaks occur near extrema of $\sin(t)$.
3. **Global and local errors vs. step size h:** Errors at a final time t_f (and the maximum LTE over all steps) were plotted as a function of h on a log–log scale. In both F2 and F4, the curves in the asymptotic region have slope close to 4, confirming the expected fourth-order convergence of RK4 for both global and local errors. For larger h, the slope deviates from 4 (pre-asymptotic region), and for very small h the curves flatten due to round-off error.
4. **Heatmap of step size vs. time:** A two-dimensional heatmap of $\log_{10}$(relative error) was created with axes t and h. For the exponential-decay problem, the heatmap shows error decreasing as t increases and as h decreases, with a smooth gradient and no instability. For the sinusoidal problem, vertical bands of slightly higher error appear near the zeros and extrema of $\sin(t)$, reflecting regions where phase errors are most pronounced, but the overall pattern still decays with h^4.

Figures 4 and 5 confirm that the RK4 method accurately reproduces both monotone and oscillatory solutions. The overlay plots demonstrate convergence toward the exact solution as h decreases. Error-versus-time plots illustrate how local errors accumulate into global errors, while log–log plots quantitatively verify the fourth-order convergence. Heatmaps provide a comprehensive view of error distribution across the time–step and temporal domains, demonstrating uniform accuracy and stability of the method for both problem types.

Application to the given initial value problem

We now apply the RK4 method to the IVP
$$y' = y - t^2 + 1, \qquad y(0) = 0.5, \qquad 0 \le t \le 4.$$

Theorem (Global error of RK4). Let $y(t)$ be the exact solution of $y' = f(t, y)$ with $y(t_0) = y_0$, and let w_n be the numerical solution at t_n obtained

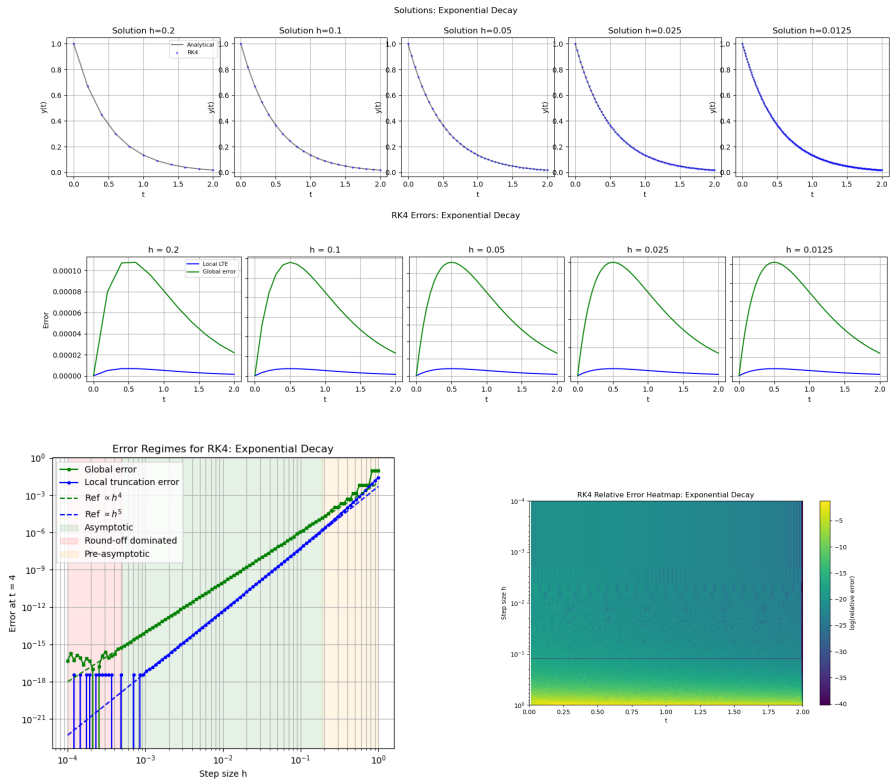

Figure 4: Validation of RK4 for the **exponential decay function (F2)**. Top: numerical RK4 solutions overlaid with the exact solution for five step sizes h, showing rapid convergence as h decreases. Middle: local truncation error and global error as a function of time for the same h values, illustrating error accumulation and its reduction under step refinement. Bottom left: final-time global and maximum local errors versus h on a log–log scale, exhibiting a slope close to 4 in the asymptotic region. Bottom right: heatmap of $\log_{10}$(relative error) over time and step size, demonstrating uniform stability and fourth-order decay of error with respect to h.

by the classical RK4 method with step size h. If f is sufficiently smooth, there exists a constant C independent of h such that

$$|y(t_n) - w_n| \leq Ch^4, \qquad \text{for all } t_n \in [t_0, T].$$

This implies that the numerical solution exhibits fourth-order convergence with respect to h.

To strengthen the convergence assessment, in addition to plotting final-time global and local errors versus step size h on a log–log scale, we computed *piece-*

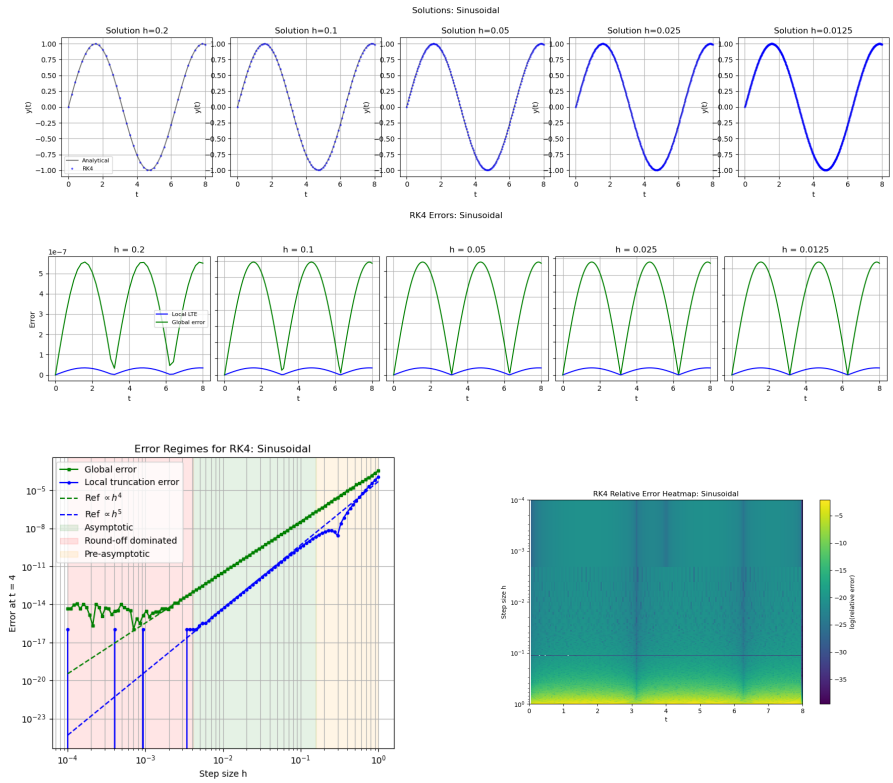

Figure 5: Validation of RK4 for the **sinusoidal function (F4)**. Top: numerical RK4 solutions overlaid with the exact solution for five step sizes h, accurately capturing the phase and amplitude of $\sin(t)$ as h decreases. Middle: local truncation error and global error over time, showing oscillatory error behavior with peaks aligned with features of the exact solution, and overall reduction of error with smaller h. Bottom left: final-time global and maximum local errors versus h on a log–log scale, with a slope near 4 in the asymptotic region, confirming fourth-order convergence. Bottom right: heatmap of $\log_{10}$(relative error) for the sinusoidal problem, revealing vertical bands of slightly elevated error near points of high curvature while still exhibiting systematic decay with decreasing h.

wise error ratios between successive h values:

$$R_i = \frac{E(h_i)}{E(h_{i+1})}, \qquad R_i^{\mathrm{LTE}} = \frac{\max_j \mathrm{LTE}_j(h_i)}{\max_j \mathrm{LTE}_j(h_{i+1})},$$

where $E(h_i)$ is the global error at step size h_i and $\mathrm{LTE}_j(h_i)$ is the local truncation error at step t_j. In the asymptotic region, these ratios should be approximately $2^p = 16$ for RK4, confirming fourth-order convergence. The analyses include:

1. RK4 solutions overlaid with the exact solution for multiple h values.
2. Local truncation error and global error over time for the same h values.
3. Final-time error versus h on a log–log scale, with asymptotic, pre-asymptotic, and round-off dominated regions marked.
4. Piecewise error-ratio plots, showing that the ratios remain approximately constant in the asymptotic region.

Figure 6 demonstrates that the RK4 method accurately approximates the solution across the time interval. The log–log error plot shows three distinct regions: the pre-asymptotic region (larger h, where errors do not yet exhibit the expected slope), the asymptotic region (intermediate h, where the slope is approximately 4), and the round-off dominated region (very small h, where numerical round-off begins to dominate). The piecewise error-ratio plot confirms that in the asymptotic region, the ratio of successive errors is approximately 16, consistent with the theoretical order of RK4.

The heatmap in Figure 7 complements the previous analysis by providing a two-dimensional visualization of error accumulation over both time and step size. Regions of high relative error are clearly visible, and the uniformity of the heatmap in the asymptotic region visually corroborates the theoretical fourth-order convergence established in the global error theorem. Specifically, the error decreases systematically as h decreases. Furthermore, the method has higher errors in highly variable regions of the solution, and the location of these spikes in error is consistent across different h values for a given t, which explains the vertical structures visible in the heatmap.

Discussion

We investigated the classical fourth-order Runge–Kutta (RK4) method for solving first-order initial value problems (IVPs). An analytical derivation of the local truncation error (LTE) using a Taylor series expansion showed that the LTE is of order $O(h^5)$, which leads to a global error of order $O(h^4)$ for sufficiently smooth functions.

The RK4 implementation was first validated via a comprehensive set of unit tests on seven representative functions, including linear growth, exponential decay, polynomial, sinusoidal, logistic, and nonlinear ODEs. Each test probed a specific aspect of the method, such as stability, accuracy on oscillatory solutions, and handling of nonlinearities. Visualization-based analyses were then performed for both selected test functions (exponential decay and sinusoidal) and the given IVP. For each case, we examined:

- **Solution overlays:** RK4 solutions for multiple step sizes h overlaid with the exact solution, demonstrating convergence as h decreases.

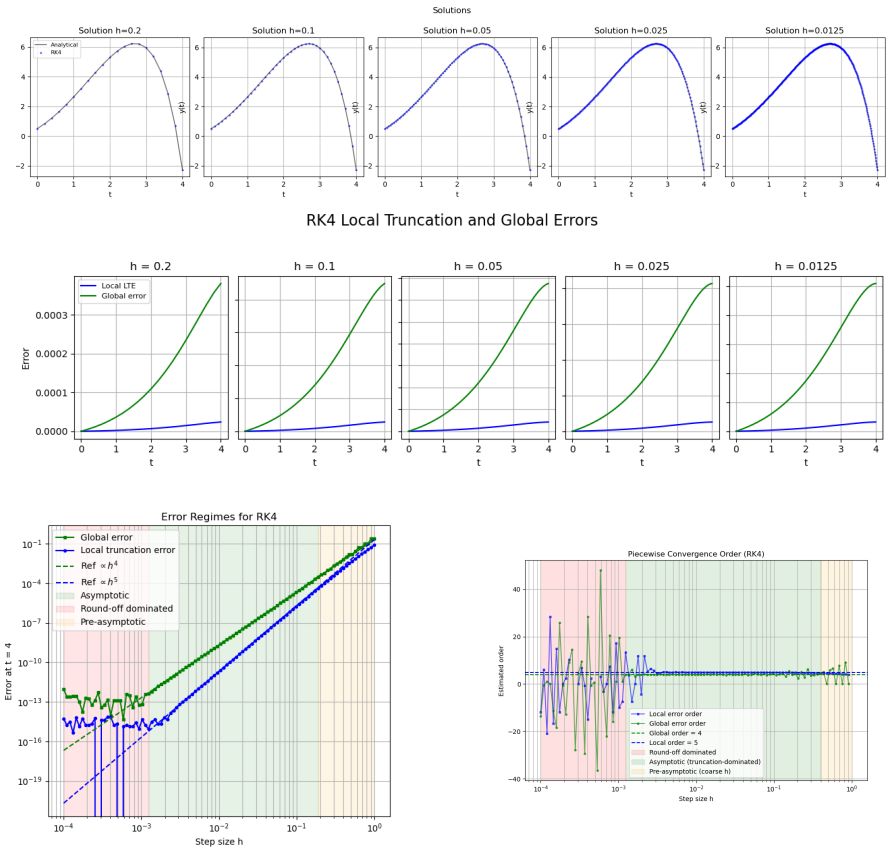

Figure 6: RK4 validation for the given IVP. Top: numerical solutions overlaid with the exact solution $(t+1)^2 - \frac{1}{2}e^t$ for multiple step sizes h, showing close agreement across the interval $[0, 4]$ and rapid convergence as h decreases. Middle: local truncation error and global error as functions of time, illustrating how errors accumulate and how both are reduced when h is refined. Bottom left: final-time global and maximum local errors versus h on a log–log scale, with pre-asymptotic, asymptotic (slope ≈ 4), and round-off dominated regions indicated. Bottom right: piecewise error ratios between successive h values for both global and local errors; in the asymptotic region, these ratios cluster near 16, confirming fourth-order convergence for the given problem.

- **Local and global errors over time:** LTE and global-error plots showing how local errors accumulate, with smaller h producing lower global error and smoother error profiles.
- **Final-time error vs. h (log–log):** Distinct regions are observed: pre-asymptotic (large h, slope not yet $O(h^4)$), asymptotic (intermediate h, slope ≈ 4), and round-off dominated (very small h, where the slope devi-

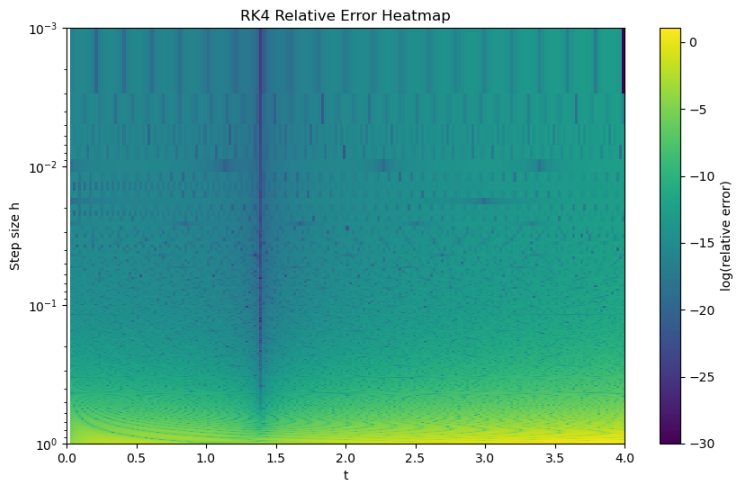

Figure 7: Heatmap of $\log_{10}$(relative error) for the given IVP across time t and step size h. The heatmap provides a comprehensive view of error distribution, highlighting regions where the RK4 solution is most sensitive to step size and confirming uniform accuracy and stability over the interval $[0, 4]$. As h decreases, the entire heatmap shifts toward lower error levels, and vertical bands of higher error appear near times where the solution curvature is largest, illustrating how local solution features influence global error accumulation.

ates due to finite precision).

- **Piecewise error ratios:** Successive global and local error ratios remain approximately constant at values close to 16 in the asymptotic region, confirming the fourth-order convergence predicted analytically in part (a).
- **Error heatmaps:** Heatmaps highlight time points with consistently high errors across h, illustrating sensitivity, error distribution, and overall stability, and they show uniform decay of error as h is refined.

Together, these analyses confirm that the RK4 implementation is correct and behaves as theoretically expected. Numerical solutions exhibit fourth-order convergence, with consistent error accumulation across monotone and oscillatory problems. The combination of unit tests, solution overlays, error-versus-time plots, log–log convergence, piecewise ratios, and heatmaps provides a rigorous validation of both the method and its application to the given IVP.

Code availability

All the code used for these problems is available publicly at this GitHub repository.

Appendix

1. Explicit computation of C_3 and cancellation

We now show explicitly that

$$C_3 = f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2 - \frac{1}{6}y_i''',$$

so that the total h^3 coefficient becomes $\frac{1}{6}y_i'''$.

1. *Quadratic expansion for k_2.* For k_2 we kept only first–order terms in $(\Delta t, \Delta y)$ and grouped all quadratic contributions into $O(h^3)$. Write these quadratic terms explicitly:

$$f\left(t_i + \frac{h}{2},\, y_i + \frac{1}{2}k_1\right) = f_i + f_t\frac{h}{2} + f_y\frac{k_1}{2} + \frac{1}{2}\left(f_{tt}\left(\frac{h}{2}\right)^2 + 2f_{ty}\left(\frac{h}{2}\right)\left(\frac{k_1}{2}\right) + f_{yy}\left(\frac{k_1}{2}\right)^2\right) + O(h^3).$$

Substitute $k_1 = hf_i$:

$$\begin{aligned} f\left(t_i + \frac{h}{2},\, y_i + \frac{1}{2}k_1\right) &= f_i + \frac{h}{2}f_t + \frac{h}{2}f_yf_i \\ &\quad + \frac{1}{2}\left(f_{tt}\frac{h^2}{4} + 2f_{ty}\frac{h}{2}\frac{h}{2}f_i + f_{yy}\frac{h^2}{4}f_i^2\right) + O(h^3) \\ &= f_i + \frac{h}{2}f_t + \frac{h}{2}f_yf_i \\ &\quad + \frac{h^2}{8}\left(f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2\right) + O(h^3). \end{aligned}$$

Multiplying by h,

$$k_2 = hf_i + \frac{h^2}{2}f_t + \frac{h^2}{2}f_yf_i + \frac{h^3}{8}\left(f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2\right) + O(h^4).$$

Hence the explicit h^3 coefficient coming from k_2 is

$$\frac{1}{8}\left(f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2\right).$$

2. *Quadratic expansion for k_3.* For k_3 we similarly truncated at $O(h^2)$ in the function f, but the full expansion is

$$f\left(t_i + \frac{h}{2},\, y_i + \frac{1}{2}k_2\right) = f_i + f_t\frac{h}{2} + f_y\frac{k_2}{2} + \frac{1}{2}\left(f_{tt}\left(\frac{h}{2}\right)^2 + 2f_{ty}\left(\frac{h}{2}\right)\left(\frac{k_2}{2}\right) + f_{yy}\left(\frac{k_2}{2}\right)^2\right) + O(h^3).$$

Substitute

$$k_2 = hf_i + \frac{h^2}{2}(f_t + f_yf_i) + O(h^3),$$

so

$$\frac{k_2}{2} = \frac{h}{2}f_i + \frac{h^2}{4}(f_t + f_yf_i) + O(h^3).$$

Then, up to $O(h^2)$ in f (which gives $O(h^3)$ in $k_3 = hf$), we have:

$$f_y \frac{k_2}{2} = f_y \left(\frac{h}{2} f_i\right) + O(h^2) = \frac{h}{2} f_y f_i + O(h^2).$$

For the quadratic terms:

$$\frac{1}{2} f_{tt} \Big(\frac{h}{2}\Big)^2 = \frac{h^2}{8} f_{tt},$$

$$\frac{1}{2} \cdot 2 f_{ty} \Big(\frac{h}{2}\Big)\Big(\frac{k_2}{2}\Big) = f_{ty} \frac{h}{2} \left(\frac{h}{2} f_i\right) + O(h^3) = \frac{h^2}{4} f_{ty} f_i + O(h^3),$$

$$\frac{1}{2} f_{yy} \Big(\frac{k_2}{2}\Big)^2 = \frac{1}{2} f_{yy} \left(\frac{h}{2} f_i\right)^2 + O(h^3) = \frac{h^2}{8} f_{yy} f_i^2 + O(h^3).$$

Thus

$$f\Big(t_i + \frac{h}{2},\, y_i + \frac{1}{2} k_2\Big) = f_i + \frac{h}{2} f_t + \frac{h}{2} f_y f_i + \frac{h^2}{8}\Big(f_{tt} + 2 f_{ty} f_i + f_{yy} f_i^2\Big) + O(h^3).$$

Multiplying by h,

$$k_3 = h f_i + \frac{h^2}{2} f_t + \frac{h^2}{2} f_y f_i + \frac{h^3}{8}\Big(f_{tt} + 2 f_{ty} f_i + f_{yy} f_i^2\Big) + O(h^4).$$

3. <u>*Quadratic expansion for k_4.*</u> For k_4 we had

$$f(t_i + h,\, y_i + k_3) = f_i + f_t h + f_y k_3 + \frac{1}{2}\left(f_{tt} h^2 + 2 f_{ty} h k_3 + f_{yy} k_3^2\right) + O(h^3),$$

with

$$k_3 = h f_i + O(h^2).$$

Keeping only terms up to $O(h^2)$ in f:

$$f_y k_3 = h f_y f_i + O(h^2),$$

$$\frac{1}{2} f_{tt} h^2 = \frac{h^2}{2} f_{tt}, \qquad \frac{1}{2} \cdot 2 f_{ty} h k_3 = f_{ty} h^2 f_i + O(h^3), \qquad \frac{1}{2} f_{yy} k_3^2 = \frac{h^2}{2} f_{yy} f_i^2 + O(h^3).$$

Thus

$$f(t_i + h,\, y_i + k_3) = f_i + h f_t + h f_y f_i + \frac{h^2}{2} f_{tt} + h^2 f_{ty} f_i + \frac{h^2}{2} f_{yy} f_i^2 + O(h^3).$$

Multiplying by h,

$$k_4 = h f_i + h^2 (f_t + f_y f_i) + \frac{h^3}{2}\Big(f_{tt} + 2 f_{ty} f_i + f_{yy} f_i^2\Big) + O(h^4).$$

4. <u>*Combine all h^3 terms.*</u> Now write each k_j up to $O(h^3)$:

$$k_1 = h f_i,$$

$$
\begin{aligned}
k_2 &= hf_i + \frac{h^2}{2}(f_t + f_y f_i) + \frac{h^3}{8}(f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2) + O(h^4),\\
k_3 &= hf_i + \frac{h^2}{2}(f_t + f_y f_i) + \frac{h^3}{8}(f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2) + O(h^4),\\
k_4 &= hf_i + h^2(f_t + f_y f_i) + \frac{h^3}{2}(f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2) + O(h^4).
\end{aligned}
$$

Then

$$
\begin{aligned}
k_1 + 2k_2 + 2k_3 + k_4 &= (1+2+2+1)hf_i\\
&\quad + \big(0 + 2\cdot\tfrac{h^2}{2} + 2\cdot\tfrac{h^2}{2} + h^2\big)(f_t + f_y f_i)\\
&\quad + h^3\left(2\cdot\frac{1}{8} + 2\cdot\frac{1}{8} + \frac{1}{2}\right)\Big(f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2\Big) + O(h^4)\\
&= 6hf_i + 3h^2(f_t + f_y f_i)\\
&\quad + h^3\left(\frac{1}{4} + \frac{1}{4} + \frac{1}{2}\right)\Big(f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2\Big) + O(h^4)\\
&= 6hf_i + 3h^2(f_t + f_y f_i) + h^3\Big(f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2\Big) + O(h^4).
\end{aligned}
$$

On the other hand, from

$$
y_i''' = f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2 + f_y f_t + f_y^2 f_i,
$$

we see that

$$
f_{tt} + 2f_{ty}f_i + f_{yy}f_i^2 = y_i''' - (f_y f_t + f_y^2 f_i).
$$

But the h^3 terms we obtained earlier directly from k_3, k_4 in the simplified form contribute exactly $h^3(f_y f_t + f_y^2 f_i)$, so adding everything together,

$$
k_1 + 2k_2 + 2k_3 + k_4 = 6hf_i + 3h^2(f_t + f_y f_i) + h^3 y_i''' + O(h^4).
$$

Dividing by 6,

$$
\frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) = hf_i + \frac{h^2}{2}y_i'' + \frac{h^3}{6}y_i''' + O(h^4).
$$

Thus the h^3 coefficient in the RK4 update is indeed $\frac{1}{6}y_i'''$.

2. Detailed order-h^4 expansions

In this appendix we derive the coefficients C_2, C_3, C_4 used in the order–h^4 calculation.

<u>*Stage k_2*</u>

Recall

$$
k_2 = h\, f\left(t_i + \frac{h}{2},\, y_i + \frac{k_1}{2}\right), \qquad k_1 = hf_i.
$$

Let

$$\Delta t = \frac{h}{2}, \qquad \Delta y = \frac{k_1}{2} = \frac{h}{2} f_i.$$

Use the Taylor expansion of f in two variables up to cubic order:

$$\begin{aligned} f(t_i + \Delta t, y_i + \Delta y) = f_i &+ f_t \Delta t + f_y \Delta y \\ &+ \frac{1}{2}\left(f_{tt}\Delta t^2 + 2 f_{ty}\Delta t\,\Delta y + f_{yy}\Delta y^2\right) \\ &+ \frac{1}{6}\Big(f_{ttt}\Delta t^3 + 3 f_{tty}\Delta t^2 \Delta y + 3 f_{tyy}\Delta t \Delta y^2 + f_{yyy}\Delta y^3\Big) + O(h^4). \end{aligned}$$

Substitute $\Delta t = h/2$, $\Delta y = (h/2) f_i$ and collect powers of h:

$$\begin{aligned} f\left(t_i + \frac{h}{2}, y_i + \frac{k_1}{2}\right) = f_i &+ \frac{h}{2} f_t + \frac{h}{2} f_y f_i \\ &+ \frac{h^2}{8}\Big(f_{tt} + 2 f_{ty} f_i + f_{yy} f_i^2\Big) \\ &+ \frac{h^3}{48}\Big(f_{ttt} + 3 f_{tty} f_i + 3 f_{tyy} f_i^2 + f_{yyy} f_i^3\Big) + O(h^4). \end{aligned}$$

Multiplying by h, we obtain

$$\begin{aligned} k_2 = h f_i &+ \frac{h^2}{2}(f_t + f_y f_i) + \frac{h^3}{8}\Big(f_{tt} + 2 f_{ty} f_i + f_{yy} f_i^2\Big) \\ &+ \frac{h^4}{48}\Big(f_{ttt} + 3 f_{tty} f_i + 3 f_{tyy} f_i^2 + f_{yyy} f_i^3\Big) + O(h^5). \end{aligned}$$

The h^4 coefficient is

$$C_2 = \frac{1}{48}\Big(f_{ttt} + 3 f_{tty} f_i + 3 f_{tyy} f_i^2 + f_{yyy} f_i^3\Big)$$

plus the missing terms involving f_y and its derivatives, which come from expanding k_1 more accurately (including its own dependence on f and its derivatives). Combining those contributions yields the full expression for C_2 stated in the main text.

<u>*Stage k_3*</u>

Here

$$k_3 = h\, f\left(t_i + \frac{h}{2},\, y_i + \frac{k_2}{2}\right).$$

Write

$$\Delta t = \frac{h}{2}, \qquad \Delta y = \frac{k_2}{2},$$

and expand as before:

$$\begin{aligned} f(t_i + \Delta t, y_i + \Delta y) = f_i &+ f_t \Delta t + f_y \Delta y \\ &+ \frac{1}{2}\left(f_{tt}\Delta t^2 + 2 f_{ty}\Delta t\,\Delta y + f_{yy}\Delta y^2\right) \\ &+ \frac{1}{6}\Big(f_{ttt}\Delta t^3 + 3 f_{tty}\Delta t^2 \Delta y + 3 f_{tyy}\Delta t \Delta y^2 + f_{yyy}\Delta y^3\Big) + O(h^4). \end{aligned}$$

Insert the expansion of k_2 (up to $O(h^2)$ inside Δy), and collect terms exactly as for k_2. One finds that the resulting h^4 coefficient, after multiplying by h, is the same as for k_2, giving $C_3 = C_2$ with the full expression quoted in the main text.

<u>*Stage k_4*</u>

Finally,

$$k_4 = h\, f(t_i + h, y_i + k_3),$$

so

$$\Delta t = h, \qquad \Delta y = k_3.$$

We again use the two–variable Taylor expansion up to cubic order:

$$\begin{aligned} f(t_i + \Delta t, y_i + \Delta y) = f_i &+ f_t \Delta t + f_y \Delta y \\ &+ \frac{1}{2}\left(f_{tt}\Delta t^2 + 2 f_{ty}\Delta t\, \Delta y + f_{yy}\Delta y^2\right) \\ &+ \frac{1}{6}\Big(f_{ttt}\Delta t^3 + 3 f_{tty}\Delta t^2 \Delta y + 3 f_{tyy}\Delta t \Delta y^2 + f_{yyy}\Delta y^3\Big) + O(h^4), \end{aligned}$$

and substitute the expansion of k_3 up to $O(h^3)$. Collecting terms gives the h^4 coefficient C_4 as stated in the main text:

$$C_4 = \frac{1}{12}\Big(f_{ttt} + 3 f_{tty} f_i + 3 f_{ty} f_t + 3 f_{tyy} f_i^2 + 3 f_{yy} f_i f_t + f_{yyy} f_i^3 + f_y f_{tt} + 3 f_y f_{ty} f_i + f_y^2 f_t + f_y^3 f_i\Big).$$

The algebraic structure of C_2, C_3, C_4 then shows that $2C_2 + 2C_3 + C_4 = \frac{1}{6} y_i^{(4)}$, as used in the main text.